

Key cryptography techniques used in blockchain

Hashing, public-private key mechanisms, and digital signatures are the key cryptographic techniques used by the blockchain community. In this case, a hash feature is implemented to offer each user the opportunity to view the blockchain in its entirety. Blockchain characteristically uses the SHA-256 hashing algorithm. Public-private key pairs are used to gain access to the information and allow transactions. Digital signatures are also used for multi-signature contracts and digital wallets on the blockchain, as well as to ratify transactions by signing them securely (offline). Figure 2 describes briefly the key algorithms used in blockchain networks.

Demonstration of the hashing function in blockchain

Hashing is a cryptographic procedure that produces a fixed length output when any data is processed through the hashing function. The Bitcoin blockchain network utilises the SHA 256 hashing function. Hash functions are designed to make it impossible to find two inputs that produce the same output. A hash function has two crucial properties: (i) a single character change will completely change the hash result, and (ii) it is a one-way function that cannot be reversed. Figure 3 gives a demonstration of the SHA 256 hash function in a blockchain network.

The working of a blockchain network

Each node in a blockchain network stores a full copy of the distributed ledger. A block in this network has four fields: a block number, a data field, a hash value, and

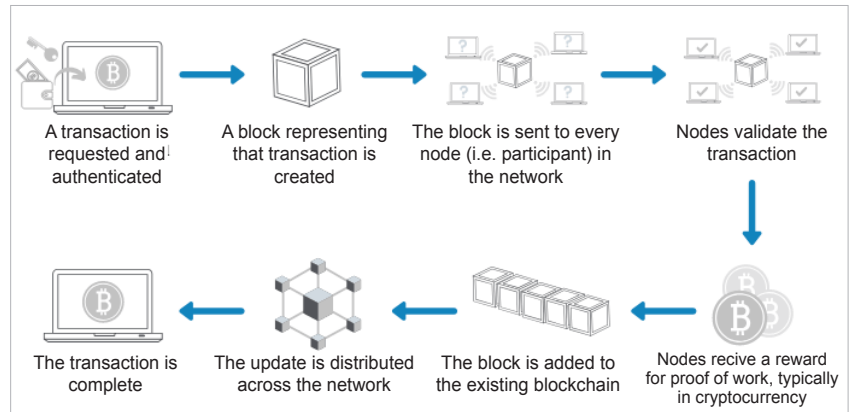


Figure 1: Block gets added to the blockchain network

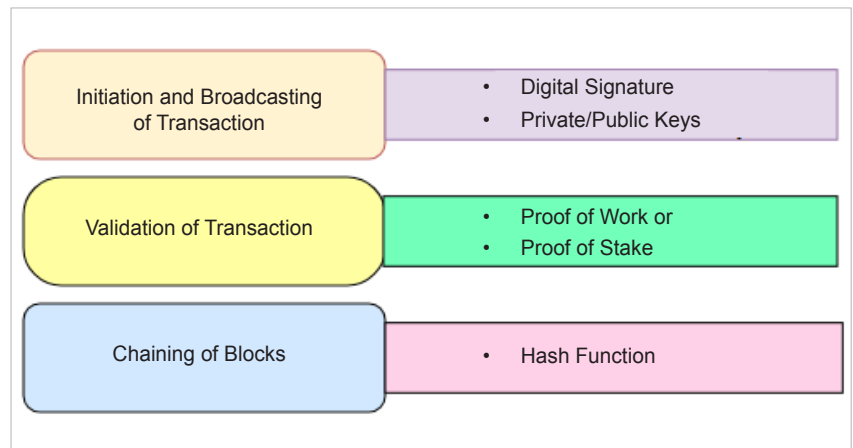


Figure 2: Key algorithms used in a blockchain network

SHA256 Hash

Data:	Open Source For You (OSFY)
Hash:	ee7bcb6978dd6f6e64cf9de46820b4cab10a69298683b07db6698bb8a52ead7d

Figure 3: Demonstration of SHA 256 hashing in blockchain

a nonce. Nonce is a random whole number that is a 32-bit (4-byte) field, which is changed by miners to make it an acceptable number for hashing the value of a block. Furthermore, the chain of blocks is formed by putting

the preceding block's hash on the current block. The field *prev* holds the hash of the previous block.

The hash value starts with four zeros, as seen in Figure 4. This specifies whether or not a block is